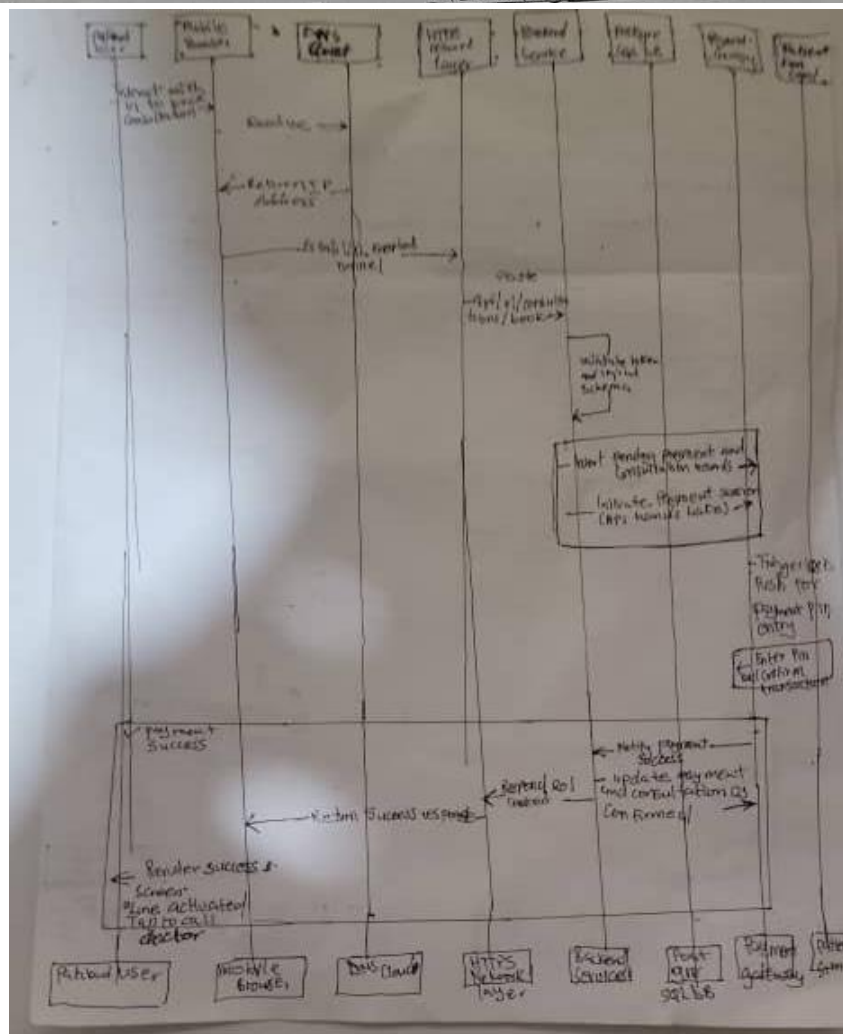


TECHCARE TEAM

Security/Privacy concern: The USSD prompt sequence
Unauthorized interception or leakage of sensitive
API session routing matrices mapping patient
Identities.



DCIT208 IA1 - Personal System Request-Flow Diagram

1. Target Client Device Profile

The user navigating this transaction sequence is assumed to be utilizing a consumer-tier computing device. In the primary context of the Akuapem North sub-district demographic, this is predominantly a mid-tier Android or iOS smartphone running modern web view runtimes, or a local computing terminal handled by staff. The hardware constraint requires that data transmission packets over the cellular network be tightly optimized for payload size, minimizing client-side memory footprint and processing overhead.

1. Client Application Environment

The team currently plans to build PluseCare as a cross-platform responsive web application optimized for mobile web browsers (such as Google Chrome, Apple Safari, or Opera Mini). The front-end framework under active consideration is React.js or Next.js, deploying a single-page application (SPA) shell architecture. By relying on a responsive web infrastructure rather than a native operating system bundle, the team ensures maximum compatibility across fragmented legacy smartphone models used by regional patients without forcing them to download large application binaries from official application stores.

2. Endpoint Uniform Resource Locator (URL) and Routing Paths

When a patient completes the input forms on their device screen, the client runtime initiates an asynchronous network call. The exact API routing infrastructure is mapped under an expressive RESTful naming convention. The target resource endpoint is: POST <https://api.plusecare.ug.edu.gh/v1/consultations/book>

This route maps directly to a specialized transaction module within our backend application layer. The path isolates the action explicitly inside the billing and scheduling engine of the clinic ecosystem, avoiding route namespace collisions with pharmacy inventory mechanisms.

3. Networking Transport Protocols

All transit data packets are wrapped using the Hypertext Transfer Protocol Secure (**HTTPS**) layer over Transport Layer Security (**TLS 1.3**). Legacy unencrypted HTTP traffic is rejected at the network firewall edge via strict transport security (HSTS) flags. The operational

transport communication protocol relies on **HTTP/2**, enabling multiplexed data streams over a single TCP connection to ensure low latency even when network conditions at the user's location experience packet drops or reduced carrier bandwidth.

4. Request Data Payload Payload Schema

The network request transmits data packaged within a lightweight JavaScript Object Notation (**JSON**) body element. The schema enforces key-value data mappings to clearly communicate the patient's request intent to the backend controller:

JSON

```
{  
  "patient_id": "ug-pat-22411279",  
  "consultation_type": "GENERAL_PRACTITIONER",  
  "facility_target": "AKUAPEM_NORTH_CENTER",  
  "payment_method": "MOBILE_MONEY",  
  "payment_phone_number": "+233201461363",  
  "telecom_provider": "VODAFONE",  
  "amount_payable": 50.00  
}
```

5. Request Authentication and Backend Validation Middleware

Before any downstream application or storage logic is executed, the inbound packet goes through a multi-tier validation middleware framework:

- **Token Verification:** The request header must include a valid JSON Web Token (Authorization: Bearer <JWT_TOKEN>) indicating the patient is logged into an authenticated session.
- **Schema Validation:** The API controller passes the JSON payload through a structural validation library (such as Zod or Joi). If any required field is absent, incorrectly typed, or if the phone number string fails to match international E.164 telemetry standards, the execution pipeline is cut off immediately.

Database Model and External API Integrations

Upon passing validation, the request triggers actions across two separate data targets:

- **External Payment Gateway API:** The system invokes a server-to-server HTTPS request to a regional telecom billing aggregator (such as Paystack or Hubtel) to trigger a Mobile Money USSD prompt on the patient's device.
- **Internal Database Layer:** The team currently plans to utilize a structured relational database system (PostgreSQL) deployed within a cloud node. The transactional record modifies rows within two explicit relation schemas:
 1. The payments table registers a tracking entity state marked as PENDING.
 2. The consultations table writes a reservation record containing foreign key links tying the patient's account ID to an available physician rota slot.

8. Server-to-Client Response Payload Architecture

If the downstream subsystems execute without errors, the application server responds to the client with an **HTTP 201 Created** status code. The response payload returns an explicitly defined JSON payload confirming state initialization:

JSON

```
{
  "success": true,
  "transaction_reference": "TXN-PC-998241",
  "booking_status": "AWAITING_PAYMENT_CONFIRMATION",
  "allocated_route_channel": "/api/v1/channels/line-0559291979",
  "timestamp": "2026-06-25T03:30:00Z"
}
```

The frontend uses this object to transition the DOM tree into a countdown loader display screen, prompting the user to check their device for the Mobile Money pin-entry screen.

9. Graceful Error Handling on Input Validation Failure

If the inbound request body fails the schema verification check, the application layer bypasses database queries to save computing resources. It returns an **HTTP 400 Bad Request** code coupled with a structured JSON error array detailing the exact validation failure:

JSON

```
{
  "success": false,
  "error_code": "INVALID_INPUT_SCHEMA",
  "message": "The provided mobile payment configuration string is malformed.",
  "invalid_fields": [
    {
      "field": "payment_phone_number",
      "reason": "Phone number must contain exactly 10 digits excluding country code."
    }
  ]
}
```

The frontend catches this payload and maps the validation errors directly onto the input fields, highlighting them in red to guide the user on what to fix.

10. Fault Tolerance Against Database Layer Downtime

If the PostgreSQL database layer is offline or experiences connection timeouts when a user submits a booking request, the application handles the failure gracefully to avoid crashing the server. The team plans to implement a circuit-breaker pattern utilizing an in-memory database cache layer (Redis).

If connection attempts to the main database fail, the API server saves the booking payload to a Redis append-only failure queue (BullMQ), returns an **HTTP 503 Service Unavailable** response to the user, and presents a user-friendly error interface: *"System experiencing heavy traffic. Your request has been queued and will process shortly."* Once database connectivity is restored, an asynchronous worker consumer empties the cache queue and synchronizes the transaction states automatically.

11. High-Concurrency Stress Scaling (500 Concurrent Hits)

If 500 patients simultaneously initiate a consultation booking at peak hours, an unoptimized application server would face database thread starvation. To survive this load, the PluseCare architecture uses several scaling strategies:

- **Asynchronous Event Loop:** Utilizing a non-blocking asynchronous I/O backend model (Node.js/Go) allows the system to process incoming connections in parallel without spawning costly OS-level execution threads.
- **Database Connection Pooling:** Implementing an active connection manager (such as PgBouncer) maintains a warm pool of reusable connections to the database, eliminating the performance penalty of creating 500 individual authentication handshakes at the database level simultaneously.
- **Rate Limiting Middleware:** The system applies a token-bucket algorithm at the proxy layer, limiting traffic to a maximum of 20 API hits per second per individual IP address to protect the app from denial-of-service stress vectors.

12. Information Security and Privacy Engineering Controls

Because this data path handles protected patient medical scheduling requests, it falls under the regulatory jurisdiction of the Ghana Data Protection Act (Act 843). The following security measures are implemented:

- **Data Masking at Rest:** Any entry containing a patient's personal identifier or consultation type flag is encrypted at rest within the table storage cells using AES-256 cryptographic algorithms.
- **Strict Transport Isolation:** Authentication data tokens are never appended within global URL string parameters where they could be logged by intermediate routing proxies. Instead, tokens are strictly injected within secure, HTTP-only cookie headers or TLS authorization request structures. This safeguards the sensitive clinical records of the Akuapem North Community Health Center from unauthorized access or network sniffing vectors.

AI DISCLOSURE

I used / did not use AI assistance. If used, the tool, purpose, prompt summary, output accepted/rejected and verification steps are documented. I understand that I must be able to explain and defend every part of this submission

I used Gemini AI to explain some basic concepts

YOUTUBE LINK

https://youtube.com/shorts/zVHngWKwvyY?is=a_jqruVrctk7HUGL

